

# Pools, Rules, and Tools: A Template-Integrated Resource Architecture

# Pools, Rules, and Tools: A Template-Integrated Resource Architecture I

**A Meta-Project Demonstrating Fonds, Rules, and Tools  
Integration in Research Software**

| Field          | Value                                                                                                                                |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Author         | Daniel Ari Friedman <sup>1</sup>                                                                                                     |
| Affiliation    | <sup>1</sup> Active Inference Institute                                                                                              |
| Correspondence | daniel@activeinference.institute                                                                                                     |
| ORCID          | 0000-0001-6232-9096                                                                                                                  |
| Version        | 1.0.0                                                                                                                                |
| Date           | 2026-07-05                                                                                                                           |
| License        | CC-BY-4.0                                                                                                                            |
| Repository     | docxology/template                                                                                                                   |
| DOI            | 10.5281/zenodo.template__pools_rules_tools                                                                                           |
| Keywords       | research software engineering, monorepo architecture, reproducibility, fonds, governance rules, tool discovery, graceful degradation |

# Author Contributions I

**Daniel Ari Friedman:** Conceptualisation, architecture design, module implementation (`fonds_reader`, `rules_applier`, `tools_invoker`, `integration`, `strong_rule_evaluator`, `figures`, `manuscript_variables`), manuscript writing (all sections), test suite design, exemplar resource authoring (`fonds`, `rules`, `tools`), validation.

# Acknowledgements I

The author thanks the Active Inference Institute for hosting the public template repository and providing the infrastructure within which this exemplar was developed. The design of the three-layer resource architecture draws inspiration from the Unix philosophy [Raymond2003art] and the enterprise application patterns documented by [Fowler2002patterns].

# Data Availability I

All source code, configuration files, and exemplar resources described in this paper are available in the public template repository at <https://github.com/docxology/template> under the `projects/templates/template_pools_rules_tools/` path. The integration pipeline is fully reproducible from source using `uv run python projects/templates/template_pools_rules_tools/scripts/02_run_integration.py` from the repository root. Generated manuscript variables are stored in `output/data/manuscript_variables.json` and injected at render time. Every figure in this manuscript, including the cover illustration, is likewise reproducible from source via `uv run python projects/templates/template_pools_rules_tools/scripts/05_generate_figures.py`, which writes directly to `manuscript/figures/`.

# Competing Interests I

The author declares no competing interests.

# Reproducibility Checklist I

This exemplar targets full computational reproducibility for every quantitative claim and figure it makes:

| Artefact                               | Regenerated by                                           | Verified by                                                                             |
|----------------------------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Integration counts (fonds/rules/tools) | scripts/02_run_integration.py                            | tests/test_integration.py                                                               |
| Manuscript variable tokens             | scripts/03_generate_manuscript.py                        | end-to-end assertion<br>Rendered PDF contains no unresolved <code>{{...}}</code> tokens |
| All 9 figures (8 content + 1 cover)    | scripts/05_generate_figures.py                           | tests/test_figures.py<br>(per-function file-existence checks)                           |
| 90% src/ line coverage                 | uv run pytest ... --cov-fail-under=90                    | CI + local pre-push gate                                                                |
| Combined PDF                           | full project pipeline, Stage 6 (4-pass xelatex + bibtex) | pdftotext scan for ?? and page-scale raster read                                        |



## Reproducibility Checklist II

A reader who clones the repository and runs the five commands above, in order, reproduces every number and image in this document without manual intervention. Every quantitative claim in this document — fond/rule/tool counts, test counts, coverage percentages, and both bar-chart figures (@fig:counts, @fig:pipeline) — is generated from the same `run_integration_demo()` call at render time. None of these numbers are hand-typed; a `test_reflects_changed_integration_result` negative control in `tests/test_manuscript_variables.py` proves the token-generation function actually tracks its source rather than emitting a fixed constant.